

PYTHON REVISION MANUAL

A Complete Beginner's Reference with Practice Questions

Covers: [Variables](#) | [Data Types](#) | [Control Flow](#) | [Functions](#) | [OOP](#) | [File I/O](#) | [Modules](#)

15 Project Ideas | Menu-Style Practice Questions | Quick Reference Tables

Python 3.x — Beginner to Intermediate

Chapter 1: Getting Started with Python

1.1 What is Python?

Python is a high-level, general-purpose programming language created by Guido van Rossum in 1991. It is known for its simple, readable syntax that resembles plain English. Python can be used for web development, data science, artificial intelligence, automation, and much more.

Why Python?

- Easy to learn — English-like syntax with minimal symbols
- Versatile — used in web, AI, data science, scripting, and automation
- Large community — thousands of free libraries available
- Interpreted — no need to compile; run code directly

1.2 Installing Python

1. Go to www.python.org and download the latest Python 3 version
2. Run the installer and check 'Add Python to PATH'
3. Open a terminal and type: `python --version`

1.3 Running Python Code

Interactive Mode (REPL)

Open terminal and type `python3` to start the interactive interpreter:

```
>>> print('Hello, World!')
Hello, World!
>>> 2 + 3
5
```

Script Mode

Write code in a `.py` file and run it from the terminal:

```
# hello.py
print('Hello, World!')

# Run from terminal:
# python3 hello.py
```

1.4 Comments and Print

```
# This is a single-line comment

"""
This is a
multi-line comment (docstring)
"""
```

```
print('Hello')          # prints text
print('Name:', 'Alice') # prints multiple values
print(10 + 5)           # prints 15
```

Chapter 2: Variables and Data Types

2.1 Variables

A variable stores a value. In Python you do not need to declare the type — it is assigned automatically.

```
name    = 'Alice'    # string
age      = 25         # integer
height  = 5.7         # float
is_active = True      # boolean

# Multiple assignment
x, y, z = 1, 2, 3
a = b = c = 0         # all equal to 0
```

2.2 Data Types

Data Type	Description & Example
int	Whole numbers: age = 25
float	Decimal numbers: price = 9.99
str	Text: name = 'Alice'
bool	True or False: is_on = True
list	Ordered, changeable: [1, 2, 3]
tuple	Ordered, unchangeable: (1, 2, 3)
dict	Key-value pairs: {'name': 'Alice'}
set	Unique values: {1, 2, 3}
None	No value / null: result = None

2.3 Type Checking and Conversion

```
x = 42
print(type(x))          # <class 'int'>

# Type conversion (casting)
int('10')               # 10
float('3.14')           # 3.14
str(100)                 # '100'
bool(0)                  # False
bool(1)                  # True
list((1, 2, 3))          # [1, 2, 3]
```

2.4 Strings in Detail

```
name = 'Alice'

# String methods
name.upper()             # 'ALICE'
```

```
name.lower()      # 'alice'
name.len()        # use len(name) -> 5
name[0]           # 'A'   (indexing)
name[1:4]         # 'lic' (slicing)
name[-1]          # 'e'   (last char)

# f-strings (recommended)
age = 25
print(f'Name: {name}, Age: {age}')
```



```
# String methods
'hello '.strip()   # 'hello'
'hello world'.split() # ['hello', 'world']
'-'.join(['a','b']) # 'a-b'
'hello'.replace('l','r') # 'herro'
'hello'.startswith('h') # True
```

Naming Rules for Variables

Must start with a letter or underscore: `_name`, `age`

Cannot start with a digit: `1age` is INVALID

Can only contain letters, digits, underscores

Case-sensitive: `Name` and `name` are different variables

Convention: use `snake_case` for variable names

Chapter 3: Operators

3.1 Arithmetic Operators

Operator	Description & Example
+	Addition: $5 + 3 = 8$
-	Subtraction: $5 - 3 = 2$
*	Multiplication: $5 * 3 = 15$
/	Division (float): $7 / 2 = 3.5$
//	Floor division (int): $7 // 2 = 3$
%	Modulus (remainder): $7 \% 2 = 1$
**	Exponent: $2 ** 3 = 8$

3.2 Comparison and Logical Operators

Operator	Description
==	Equal to: $5 == 5 \rightarrow \text{True}$
!=	Not equal: $5 != 3 \rightarrow \text{True}$
>	Greater than: $5 > 3 \rightarrow \text{True}$
<	Less than: $3 < 5 \rightarrow \text{True}$
>=	Greater or equal
<=	Less or equal
and	Both must be True: $\text{True and False} \rightarrow \text{False}$
or	At least one True: $\text{True or False} \rightarrow \text{True}$
not	Reverses: $\text{not True} \rightarrow \text{False}$
in	Value in collection: $'a' \text{ in } 'cat' \rightarrow \text{True}$

3.3 Assignment Operators

```
x = 10
x += 5    # x = x + 5  -> 15
x -= 3    # x = x - 3  -> 12
x *= 2    # x = x * 2   -> 24
x //= 4   # x = x // 4  -> 6
x **= 2   # x = x ** 2  -> 36
```

Chapter 4: Control Flow

4.1 if / elif / else

```
score = 75

if score >= 90:
    print('Grade A')
elif score >= 75:
    print('Grade B')
elif score >= 60:
    print('Grade C')
else:
    print('Fail')
```

Indentation Rule

Python uses indentation (4 spaces) to define blocks of code.

Do NOT mix tabs and spaces — this causes IndentationError.

Every if, for, while, def, class block must be indented.

4.2 for Loops

```
# Loop over a list
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)

# Loop with range
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

for i in range(1, 6):   # 1, 2, 3, 4, 5
    print(i)

for i in range(0, 10, 2): # 0, 2, 4, 6, 8
    print(i)

# Loop with index using enumerate
for i, fruit in enumerate(fruits):
    print(i, fruit)      # 0 apple, 1 banana ...
```

4.3 while Loops

```
count = 0
while count < 5:
    print(count)
    count += 1

# while with user input
while True:
    answer = input('Enter q to quit: ')
    if answer == 'q':
        break
```

4.4 Loop Control: break, continue, pass

Keyword	Purpose & Example
break	Stop the loop entirely
continue	Skip to the next iteration
pass	Do nothing (placeholder)

```
for i in range(10):
    if i == 5:
        break      # stops at 5
    if i % 2 == 0:
        continue   # skips even numbers
    print(i)        # prints 1, 3
```

Chapter 5: Functions

5.1 Defining and Calling Functions

```
# Define a function
def greet(name):
    print('Hello, ' + name + '!')

# Call it
greet('Alice')    # Hello, Alice!
greet('Bob')     # Hello, Bob!
```

5.2 Parameters and Return Values

```
def add(a, b):
    return a + b

result = add(3, 5)    # result = 8

# Default parameter values
def greet(name, greeting='Hello'):
    return greeting + ', ' + name

greet('Alice')        # Hello, Alice
greet('Bob', 'Hi')    # Hi, Bob

# Multiple return values
def min_max(numbers):
    return min(numbers), max(numbers)

low, high = min_max([3, 1, 7, 2])
```

5.3 *args and **kwargs

```
# *args: variable number of positional arguments
def total(*args):
    return sum(args)

total(1, 2, 3)        # 6
total(10, 20, 30)     # 60

# **kwargs: variable keyword arguments
def show_info(**kwargs):
    for key, value in kwargs.items():
        print(key + ': ' + str(value))

show_info(name='Alice', age=25)
```

5.4 Lambda Functions

```
# Regular function
def square(x):
    return x * x

# Same as lambda
```

```
square = lambda x: x * x
print(square(5))    # 25

# Lambda with sorted()
students = [('Alice', 85), ('Bob', 92), ('Charlie', 78)]
students.sort(key=lambda s: s[1])    # sort by score
```

5.5 Scope: Local vs Global

```
x = 10    # global variable

def example():
    x = 20    # local variable (different from global x)
    print(x) # 20

example()
print(x)     # 10 (global unchanged)

# To modify global inside function:
def change():
    global x
    x = 99
```

Chapter 6: Lists, Tuples, Sets, and Dictionaries

6.1 Lists

```
fruits = ['apple', 'banana', 'cherry']

# Accessing
fruits[0]      # 'apple'
fruits[-1]     # 'cherry'
fruits[1:3]    # ['banana', 'cherry']

# Modifying
fruits.append('mango')    # add to end
fruits.insert(1, 'kiwi')  # insert at index 1
fruits.remove('banana')   # remove by value
fruits.pop()              # remove last item
fruits.pop(0)             # remove at index 0

# Useful methods
fruits.sort()              # sort in place
fruits.reverse()           # reverse in place
len(fruits)                # length
'apple' in fruits          # True / False
fruits.count('apple')      # how many 'apple'
fruits.index('cherry')     # index of 'cherry'
```

6.2 List Comprehensions

```
# Traditional way
squares = []
for x in range(1, 6):
    squares.append(x ** 2)

# List comprehension (same result, one line)
squares = [x ** 2 for x in range(1, 6)]
# [1, 4, 9, 16, 25]

# With condition
evens = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]
```

6.3 Tuples

```
point = (3, 7)          # cannot be changed
x, y = point             # unpacking
print(x, y)              # 3 7

# Single element tuple
single = (42,)           # note the comma

# Tuples as dictionary keys (lists cannot be keys)
coordinates = {(0, 0): 'origin', (1, 0): 'right'}
```

6.4 Dictionaries

```
student = {
    'name': 'Alice',
    'age': 25,
    'grade': 'A'
}

# Accessing
student['name']          # 'Alice'
student.get('age', 0)    # 25 (safe, default 0)

# Modifying
student['age'] = 26       # update
student['city'] = 'Delhi' # add new key
del student['grade']      # delete key

# Looping
for key in student:      # keys
    print(key)
for key, value in student.items():
    print(key, ': ', value)

# Useful methods
student.keys() # dict_keys(['name', 'age', ...])
student.values() # dict_values(['Alice', 26, ...])
'name' in student # True
```

6.5 Sets

```
colors = {'red', 'green', 'blue'}

colors.add('yellow')    # add element
colors.remove('red')     # remove (raises error if missing)
colors.discard('red')    # remove safely

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
a | b # union: {1, 2, 3, 4, 5, 6}
a & b # intersection: {3, 4}
a - b # difference: {1, 2}
```

Chapter 7: File Handling

7.1 Reading Files

```
# Read entire file
with open('file.txt', 'r') as f:
    content = f.read()

# Read line by line
with open('file.txt', 'r') as f:
    for line in f:
        print(line.strip())

# Read all lines as a list
with open('file.txt', 'r') as f:
    lines = f.readlines()
```

7.2 Writing Files

```
# Write (overwrites existing content)
with open('output.txt', 'w') as f:
    f.write('Hello, World!\n')
    f.write('Second line')

# Append (adds to existing content)
with open('log.txt', 'a') as f:
    f.write('New log entry\n')
```

7.3 File Modes

Mode	Description
'r'	Read (default). Error if file not found.
'w'	Write. Creates file or overwrites existing.
'a'	Append. Adds to end of file.
'x'	Create. Error if file already exists.
'b'	Binary mode (combine: 'rb', 'wb').

7.4 Working with JSON

```
import json

# Python dict to JSON file
data = {'name': 'Alice', 'age': 25}
with open('data.json', 'w') as f:
    json.dump(data, f, indent=2)

# JSON file to Python dict
with open('data.json', 'r') as f:
    loaded = json.load(f)
print(loaded['name']) # Alice
```

7.5 The os Module

```
import os

os.getcwd()           # current directory
os.listdir('.')        # list files in folder
os.path.exists('file.txt') # True / False
os.path.join('folder', 'file.txt') # safe path
os.makedirs('new_folder', exist_ok=True)
os.rename('old.txt', 'new.txt')
os.remove('file.txt')  # delete a file
```

Chapter 8: Error Handling

8.1 try / except

```
try:
    x = int(input('Enter a number: '))
    result = 10 / x
    print('Result:', result)
except ValueError:
    print('Error: Please enter a valid integer.')
except ZeroDivisionError:
    print('Error: Cannot divide by zero.')
except Exception as e:
    print('Unexpected error:', e)
else:
    print('No errors occurred.')
finally:
    print('This always runs.')
```

8.2 Common Exceptions

Exception	When it occurs
ValueError	Wrong value type: int('abc')
TypeError	Wrong type used in operation
ZeroDivisionError	Division by zero: 5 / 0
IndexError	List index out of range: list[100]
KeyError	Dict key does not exist: d['missing']
FileNotFoundError	File does not exist on disk
AttributeError	Object does not have the attribute
NameError	Variable not defined

8.3 Raising Exceptions

```
def get_age(age):
    if age < 0:
        raise ValueError('Age cannot be negative')
    return age

try:
    get_age(-5)
except ValueError as e:
    print('Error:', e)
```

Chapter 9: Object-Oriented Programming (OOP)

9.1 Classes and Objects

```
class Dog:
    # Class attribute (shared by all dogs)
    species = 'Canis familiaris'

    # Constructor (__init__ runs when object is created)
    def __init__(self, name, breed, age):
        self.name = name    # instance attribute
        self.breed = breed
        self.age = age

    # Method (function inside class)
    def bark(self):
        return self.name + ' says: Woof!'

    def describe(self):
        return self.name + ' is a ' + self.breed

# Create objects
dog1 = Dog('Buddy', 'Labrador', 3)
dog2 = Dog('Max', 'Poodle', 5)

print(dog1.bark())    # Buddy says: Woof!
print(dog2.describe()) # Max is a Poodle
print(Dog.species)    # Canis familiaris
```

9.2 Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return 'Some sound'

# Dog inherits from Animal
class Dog(Animal):
    def speak(self):    # override parent method
        return self.name + ' says Woof!'

class Cat(Animal):
    def speak(self):
        return self.name + ' says Meow!'

animals = [Dog('Buddy'), Cat('Whiskers')]
for a in animals:
    print(a.speak())
```

9.3 Special Methods (Dunder Methods)

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages
```

```
def __str__(self):      # print(book) output
    return self.title + ' (' + str(self.pages) + ' pages)'

def __len__(self):      # len(book) output
    return self.pages

def __repr__(self):     # developer representation
    return 'Book(' + self.title + ')'
```

```
b = Book('Python Guide', 300)
print(b)                # Python Guide (300 pages)
print(len(b))           # 300
```

Chapter 10: Useful Python Modules

10.1 Standard Library Overview

Module	What it does
random	Random numbers, choices, shuffles
math	Math functions: sqrt, ceil, floor, pi
datetime	Dates and times
os	File and folder operations
sys	System-level operations
json	Read and write JSON data
re	Regular expressions (pattern matching)
time	Sleep, measure time
string	String constants and utilities
collections	Counter, defaultdict, deque

10.2 random Module

```
import random

random.randint(1, 10)      # random int 1 to 10
random.random()            # float between 0.0 and 1.0
random.choice(['a','b','c']) # random item from list
random.shuffle([1,2,3,4])  # shuffle list in place
random.sample(range(100), 5) # 5 unique random numbers
```

10.3 datetime Module

```
from datetime import datetime

now = datetime.now()
print(now)                # 2024-01-15 14:30:00
print(now.strftime('%d/%m/%Y')) # 15/01/2024
print(now.strftime('%H:%M:%S')) # 14:30:00
print(now.year, now.month, now.day) # 2024 1 15
```

10.4 math Module

```
import math

math.sqrt(25)      # 5.0
math.ceil(4.2)     # 5 (round up)
math.floor(4.8)    # 4 (round down)
math.pi           # 3.141592...
math.factorial(5)  # 120
```

```
math.pow(2, 10)    # 1024.0  
math.log(100, 10)  # 2.0
```

Chapter 11: Practice Questions

11.1 Multiple Choice Questions

Instructions

Circle or write the letter of the best answer.

Each question has one correct answer.

Section A: Variables and Data Types

1. What is the output of: `print(type(3.14))`?

- A) <class 'int'>
- B) <class 'float'>
- C) <class 'str'>
- D) <class 'double'>

Answer: _____

2. Which of the following is a valid variable name?

- A) 1name
- B) my-name
- C) my_name
- D) my name

Answer: _____

3. What does `bool(0)` return?

- A) 0
- B) None
- C) True
- D) False

Answer: _____

4. What is the output of: `len('Python')`?

- A) 5
- B) 6
- C) 7
- D) Error

Answer: _____

5. Which data type is immutable (cannot be changed)?

- A) list
- B) dict
- C) tuple
- D) set

Answer: _____

Section B: Control Flow

1. What does 'break' do inside a loop?

- A) Skips to next iteration
- B) Exits the loop entirely

- C) Does nothing
- D) Causes an error

Answer: _____

2. What is the output of: `range(2, 8, 2)`?

- A) [2, 4, 6, 8]
- B) [2, 4, 6]
- C) [2, 3, 4, 5, 6, 7]
- D) [0, 2, 4, 6]

Answer: _____

3. Which keyword is used to handle 'no condition matched' in if-elif?

- A) default
- B) otherwise
- C) else
- D) final

Answer: _____

Section C: Functions

1. What keyword is used to return a value from a function?

- A) send
- B) output
- C) give
- D) return

Answer: _____

2. What is the output of: `square = lambda x: x**2` then `square(4)`?

- A) 8
- B) 16
- C) 4
- D) Error

Answer: _____

3. A function with `*args` can accept:

- A) Only 2 arguments
- B) Exactly 3 arguments
- C) Any number of positional arguments
- D) Keyword arguments only

Answer: _____

Section D: Lists and Dictionaries

1. Which method adds an item to the end of a list?

- A) `add()`
- B) `push()`
- C) `append()`
- D) `insert()`

Answer: _____

2. What does `fruits[1:3]` return for `fruits = ['a','b','c','d']`?

- A) ['a','b']

- B) ['b','c']
- C) ['b','c','d']
- D) ['a','b','c']

Answer: _____

3. How do you safely get a dictionary value without raising a KeyError?

- A) d['key']
- B) d.fetch('key')
- C) d.get('key')
- D) d.find('key')

Answer: _____

11.2 Fill in the Blank

1. The keyword used to define a function is _____.
2. To open a file for reading in Python, use `open('file.txt', _____)`.
3. A _____ is an unordered collection of unique items.
4. The method that removes and returns the last item of a list is _____.
5. To convert the string '42' to an integer, write: _____('42').
6. In a try/except block, the _____ block always executes.
7. The operator `//` performs _____ division.
8. To inherit from a parent class, write: `class Child(_____)`:
9. The built-in function to find the length of a list is _____.
10. An f-string is written as: _____'Hello, {name}'.

11.3 Code Output Questions

What is the output of each code snippet? Write your answer in the box.

Q1:

```
x = 5 y = 3 print(x ** y)
```

Your Answer:

Q2:

```
nums = [1, 2, 3, 4, 5] print(nums[1:4])
```

Your Answer:

Q3:

```
for i in range(1, 6):    if i % 2 == 0:        print(i)
```

Your Answer:

Q4:

```
d = {'a':1, 'b':2, 'c':3} print(len(d))
```

Your Answer:

Q5:

```
x = 10 def f():    x = 20    print(x) f() print(x)
```

Your Answer:

11.4 Menu-Style Exercises

Each exercise below presents a menu. Read it and answer the questions that follow.

Exercise 1: Calculator Menu

```
def show_menu():
    print('1. Add')
    print('2. Subtract')
    print('3. Exit')
    return input('Choice: ')

while True:
    choice = show_menu()
    if choice == '1':
        a = float(input('First: '))
        b = float(input('Second: '))
        print('Result:', a + b)
    elif choice == '2':
        a = float(input('First: '))
        b = float(input('Second: '))
        print('Result:', a - b)
    elif choice == '3':
        break
```

Questions:

1. What happens if the user enters '4'?
2. How would you add a Multiply option?
3. Why is input() wrapped in float()?
4. What does 'break' do in this context?

Exercise 2: Fix the Bug

Each code snippet has one bug. Find and fix it.

Bug 1:

```
def greet(name)
    print('Hello, ' + name)
```

Hint: Check the function definition

Fix:

Bug 2:

```
fruits = ['apple', 'banana']
print(fruits[2])
```

Hint: Check the index

Fix:

Bug 3:

```
for i in range(5):
    print(i)
```

Hint: Check indentation

Fix:

Bug 4:

```
x = '10'  
y = 5  
print(x + y)
```

Hint: Check data types

Fix:

Chapter 12: Quick Reference — Cheat Sheet

12.1 Python Syntax Cheat Sheet

Concept	Syntax	Notes
Variable	<code>x = 10</code>	No type declaration needed
String	<code>s = 'hello'</code>	Single or double quotes
f-string	<code>f'Hi {name}'</code>	Embed variables in strings
List	<code>[1, 2, 3]</code>	Ordered, changeable
Tuple	<code>(1, 2, 3)</code>	Ordered, unchangeable
Dict	<code>{'key': 'value'}</code>	Key-value pairs
Set	<code>{1, 2, 3}</code>	Unique values only
if	<code>if x > 0: ...</code>	Colon + indented block
for loop	<code>for i in range(5): ...</code>	Iterates over sequence
while loop	<code>while x > 0: ...</code>	Runs while condition true
function	<code>def f(x): return x</code>	def keyword required
lambda	<code>lambda x: x * 2</code>	One-line anonymous function
class	<code>class Dog: ...</code>	Blueprint for objects
try/except	<code>try: ... except E: ...</code>	Error handling
import	<code>import math</code>	Load a module
from import	<code>from math import sqrt</code>	Import specific function

12.2 String Methods Quick Reference

Method	Example and Result
<code>upper()</code>	<code>'hello'.upper() -> 'HELLO'</code>
<code>lower()</code>	<code>'HELLO'.lower() -> 'hello'</code>
<code>strip()</code>	<code>' hi '.strip() -> 'hi'</code>
<code>split()</code>	<code>'a,b,c'.split(',') -> ['a','b','c']</code>
<code>join()</code>	<code>'-'.join(['a','b']) -> 'a-b'</code>
<code>replace()</code>	<code>'cat'.replace('c','b') -> 'bat'</code>
<code>find()</code>	<code>'hello'.find('e') -> 1</code>
<code>startswith()</code>	<code>'Python'.startswith('Py') -> True</code>
<code>endswith()</code>	<code>'file.py'.endswith('.py') -> True</code>
<code>count()</code>	<code>'hello'.count('l') -> 2</code>

Method	Example and Result
<code>isdigit()</code>	<code>'123'.isdigit() -> True</code>
<code>isalpha()</code>	<code>'abc'.isalpha() -> True</code>

Chapter 13: Project Code Summaries

Below is a brief description of each of the 15 Python projects in this bundle, along with key concepts used in each.

Project	Key Concepts	Difficulty
01. Calculator	Functions, dict, exception handling	Beginner
02. To-Do List Manager	JSON, lists, file I/O, CRUD logic	Beginner
03. Number Guessing	random, while loop, scoring logic	Beginner
04. Password Generator	random, string module, functions	Beginner
05. Dice Roller	random, ASCII art, statistics	Beginner
06. Weather App	requests, API, JSON parsing	Beginner+
07. Quiz Game	dict, lists, scoring, random.sample()	Beginner
08. Simple Chatbot	if/elif, dict, string matching	Beginner
09. Unit Converter	functions, dict, nested menus	Beginner
10. Alarm Clock	datetime, threading, time module	Beginner+
11. Expense Tracker	JSON, filtering, grouping, file I/O	Intermediate
12. Markdown to HTML	re module, string methods, file I/O	Intermediate
13. Contact Book	dict, JSON, search, CRUD operations	Beginner+
14. Hangman Game	random, ASCII art, sets, game loop	Beginner+
15. File Organizer	os, shutil, file extensions, directories	Intermediate

Tips for Beginners

Start by reading the code top to bottom and understanding each function.

Run the program and test every menu option before modifying anything.

Add one small feature at a time — for example, add a 'search' feature to the to-do list.

When you get an error, read the error message carefully — Python tells you exactly what went wrong.

Practice by changing variable names, adding new options, or combining two projects.

